

GCI

Supervised Learning

 松尾・岩澤研究室
MATSUO-IWASAWA LAB UTOKYO

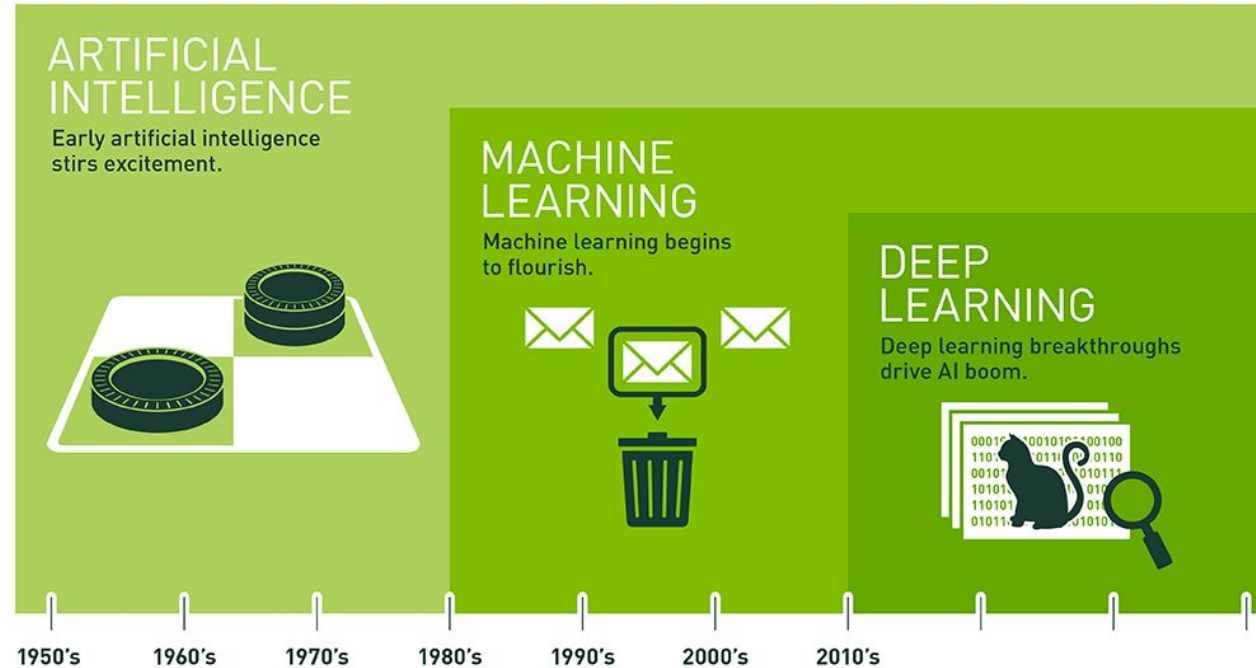
- What is Supervised Learning?
- Supervised Learning Algorithms
- Overfitting and Regularization
- Evaluating Models
- Hyperparameter Tuning

What is Supervised Learning?

AI? Machine Learning? Deep Learning?



- **Artificial Intelligence (AI)**: A field of research that aims to create an artificial human-level intelligence
- **Machine Learning**: Method to train a program to learn patterns from data
- **Deep Learning**: Method of machine learning that uses neural networks



Reference: <https://blogs.nvidia.com/blog/2016/07/29/whats-difference-artificial-intelligence-machine-learning-deep-learning-ai/>

What is Machine Learning?



- Machine Learning: Method to train a program to learn patterns from data
- By learning, the program can **make predictions from the learned patterns**

Loading Data



Cap size 10cm 5cm 7cm 4cm

Stalk width 1cm 2cm 5cm 2cm

Poisonous? No Yes No Yes

“Sorry, I was just joking...”

“Free entry in 2 a wkly comp to win FA Cup final tkts”

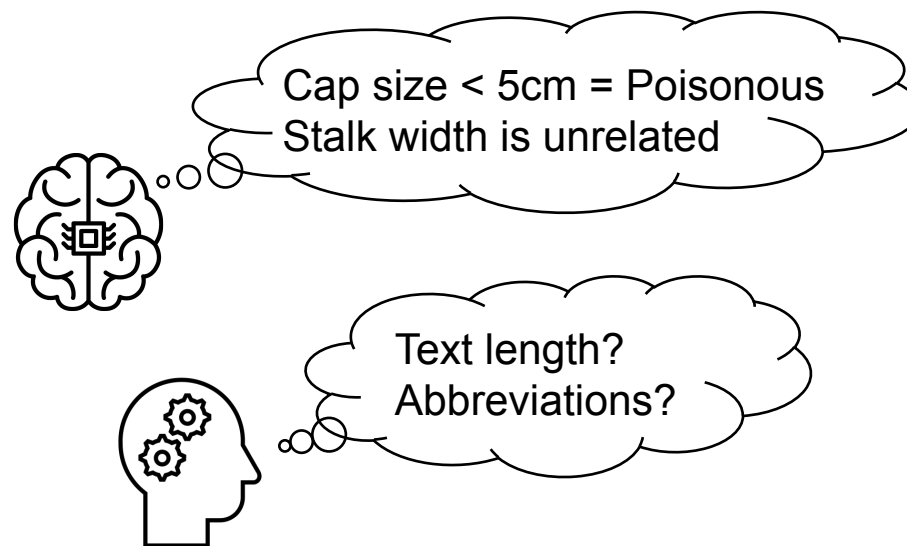
“Nah I don't think he goes to usf”

Not spam

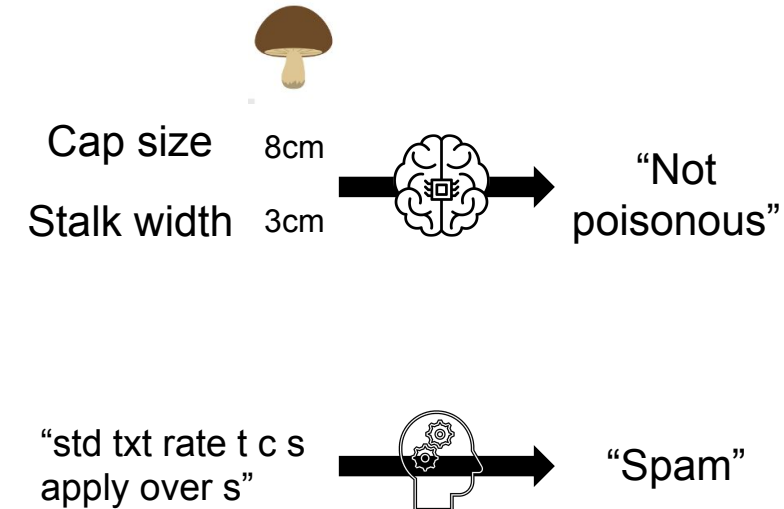
Spam

Not spam

Learn from Data



Making Predictions



Textbooks, Past exams



How do I solve these problems?

Study for exam

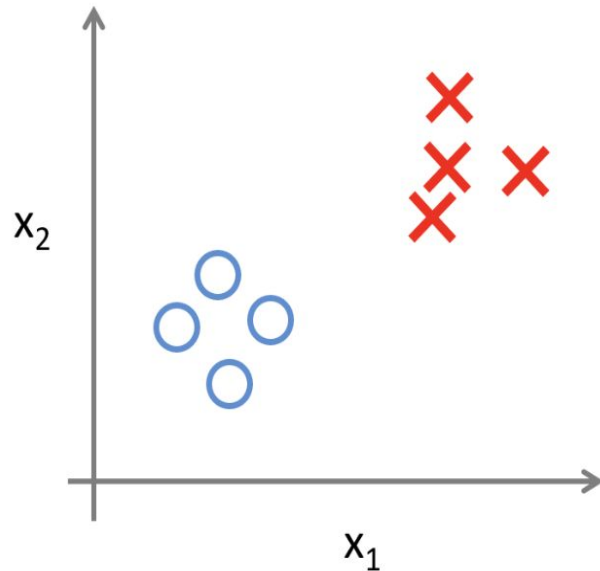


Actual exam

Types of Machine Learning

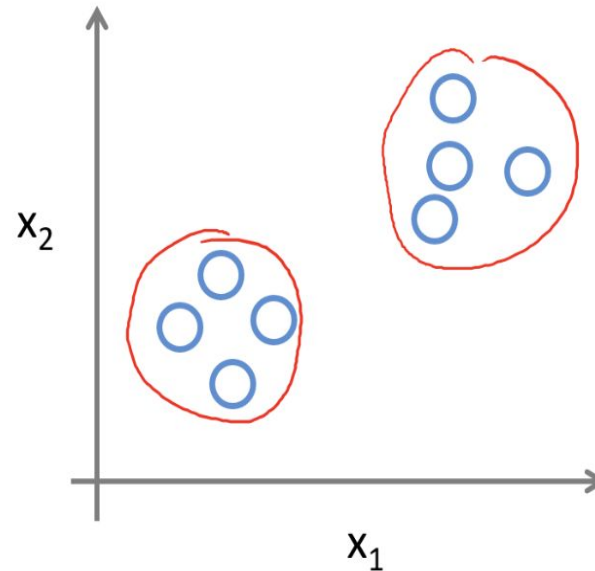


Supervised Learning



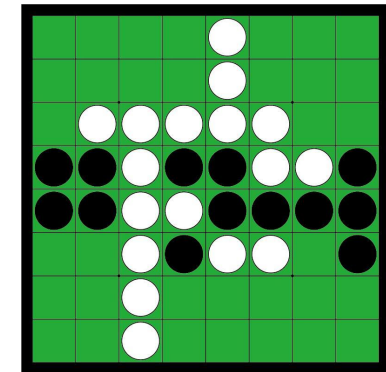
- Have ground-truth values
- Learn the relationship between input and output

Unsupervised Learning



- No ground-truth values
- Find patterns in the data

Reinforcement Learning

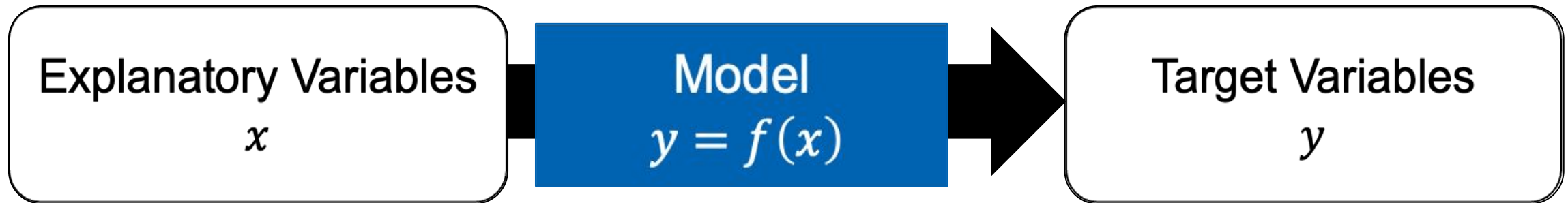


- Learn to maximize reward

Reference: <https://lakshaysuri.wordpress.com/2017/03/19/machine-learning-supervised-vs-unsupervised-learning/>

What is Supervised Learning?

- Motivation: Learn the relationship f between the input x and output y
 - Input: **Explanatory variables**
 - Output: **Target variables**
 - Relationship: **Model** (some machine learning algorithm)



- Examples
 - Image recognition, fraud detection, trend prediction, etc.

Types of Machine Learning



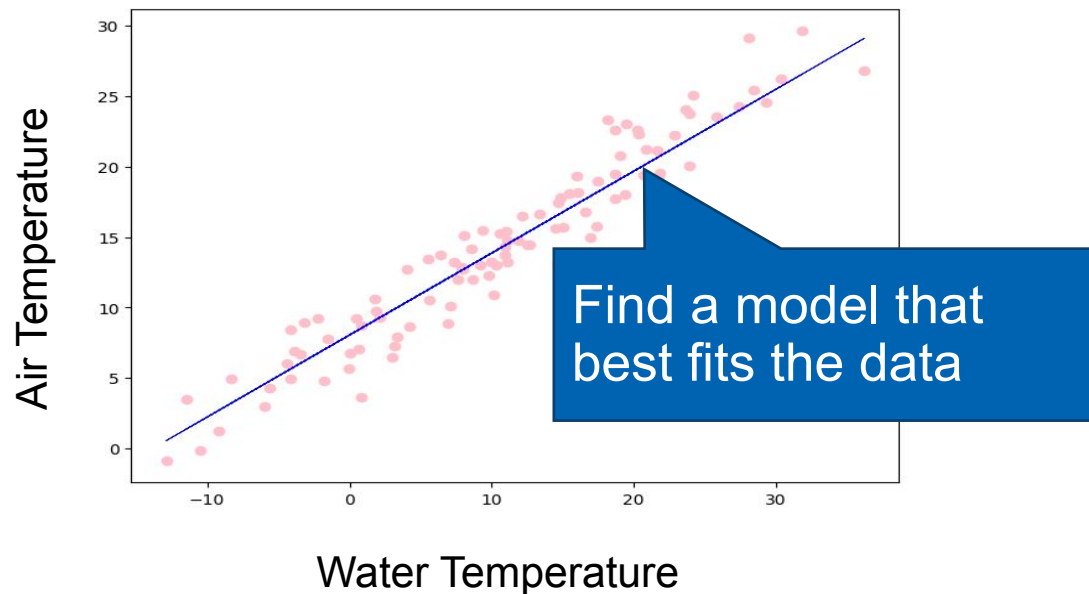
There are two types of supervised learning problems: regression and classification.

- **Regression**

- Predict continuous values

- **Examples**

- Predict air temperatures from water temperature



- **Classification**

- Predict discrete values (labels)

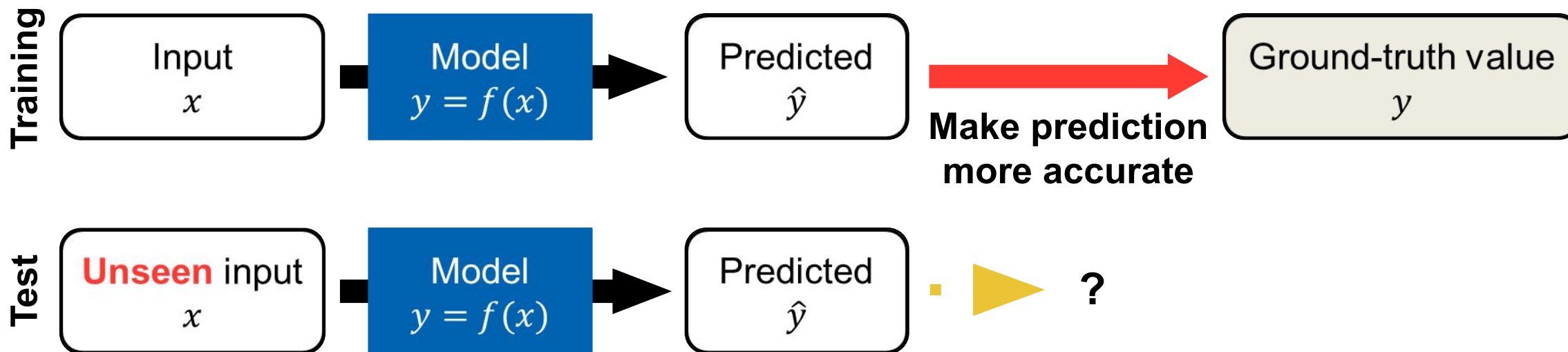
- **Examples**

- Predict if an image is about a dog or cat



Goal of Supervised Learning

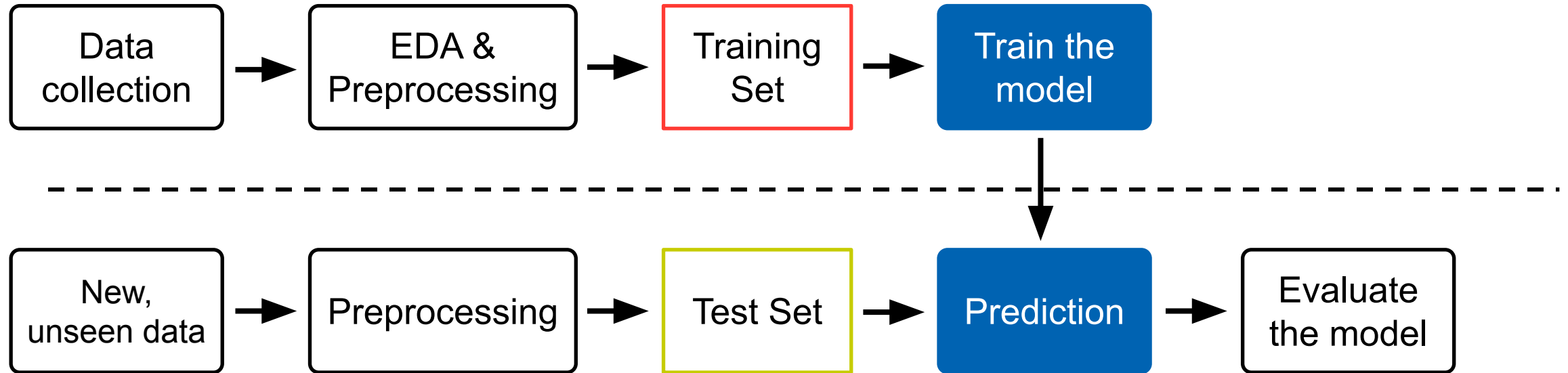
The objective is to find a model that accurately predict on new, unseen data.



- **Generalization**

- If a model can accurately predict on an unseen dataset, we say that it has successfully generalized the trends and patterns in the data.

Supervised Learning Workflow (minimal)



Supervised Learning Algorithms

Linear Regression



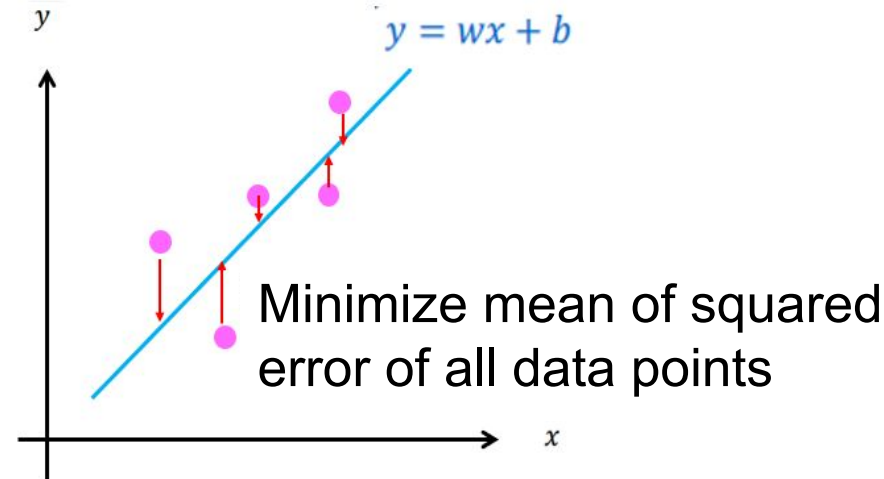
Assume a linear relationship between input/output and find best fitting line.

- Purpose: regression
- Model

$$f(\mathbf{x}) = w_1x_1 + w_2x_2 + \cdots w_Mx_M + b = \mathbf{w}\mathbf{x} + b$$

weights **bias**
↓ ↓

- **Find w, b that best minimizes the squared error, $\|\hat{y} - y\|^2$** , between predicted and ground-truth value
- Example ($M = 1$)



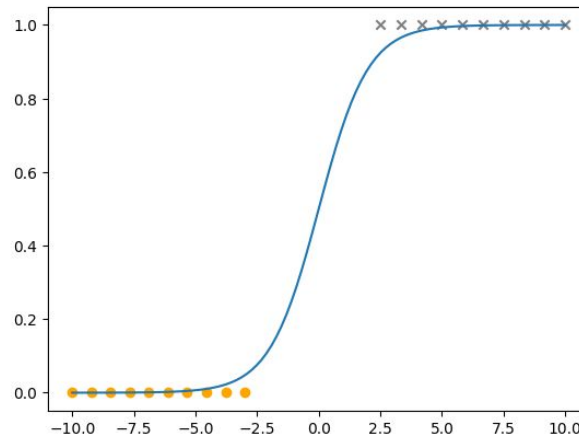
Logistic regression



Linear regression model extended for classification problem.

- Purpose: classification (*despite being called regression)
- Model

$$f(\mathbf{x}) = \sigma(\mathbf{w}\mathbf{x} + b) = \frac{1}{1 + e^{-(w_1x_1 + w_2x_2 + \dots + w_Mx_M + b)}}$$



The **sigmoid function** converts the output so that it can predict probabilities

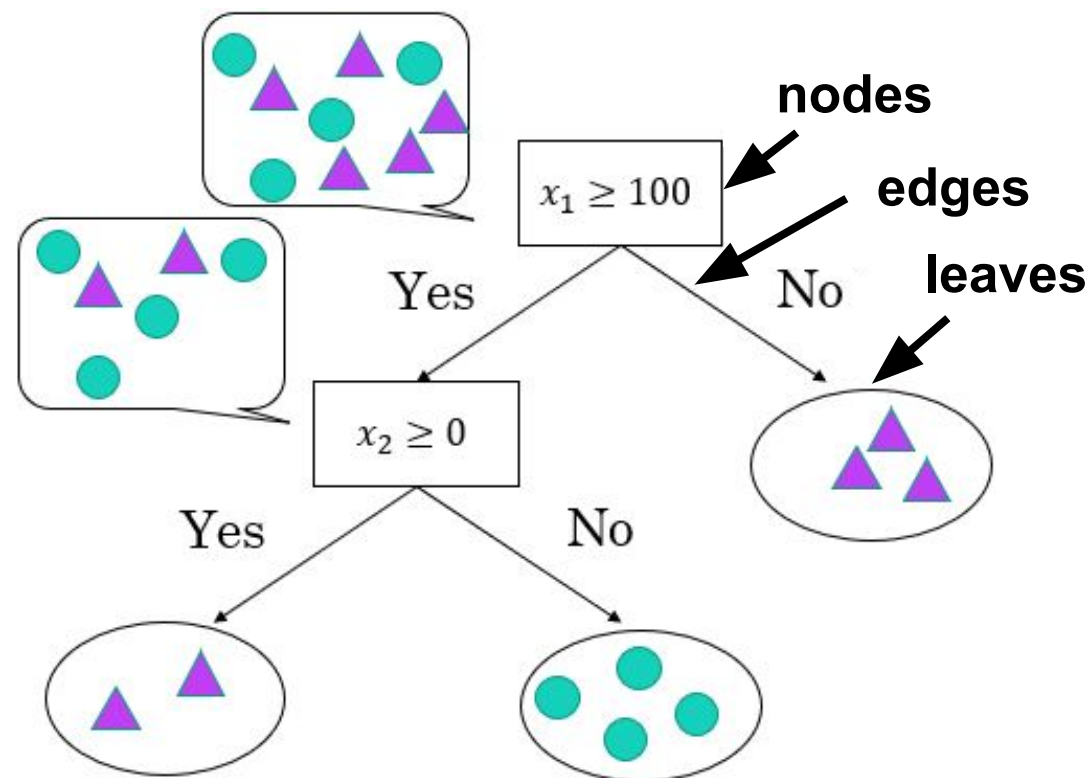
- **Find w, b that best minimizes the cross-entropy loss**, between predicted and ground-truth value

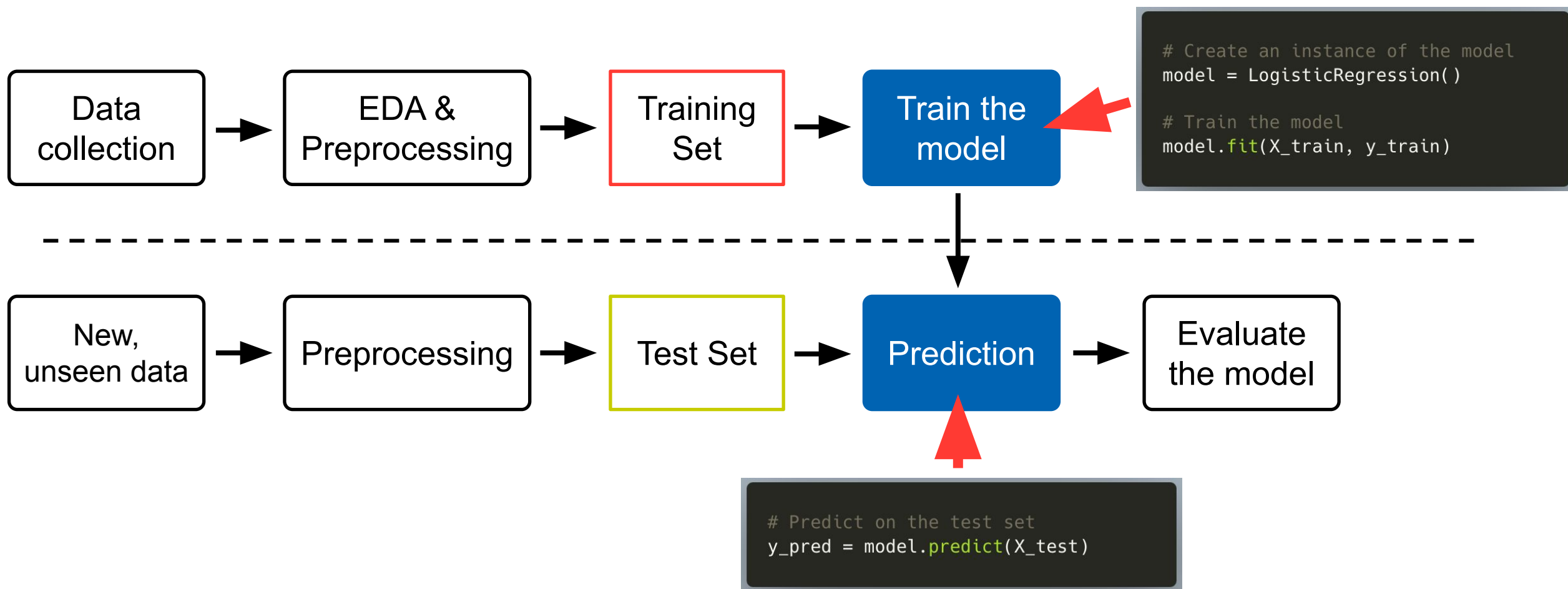
Decision Trees



Tree-based algorithm that can be used for both regression and classification.

- Purpose: regression & classification
- Model
 - Predicts using a **tree structure**
 - Nodes: Decision points
 - Edges: Paths that split data based on the decision at a node
 - Leaves: Assigns final prediction
 - Each “decision” is decided by calculating **information gain**, the difference in the degree of mixing of the data before and after splitting

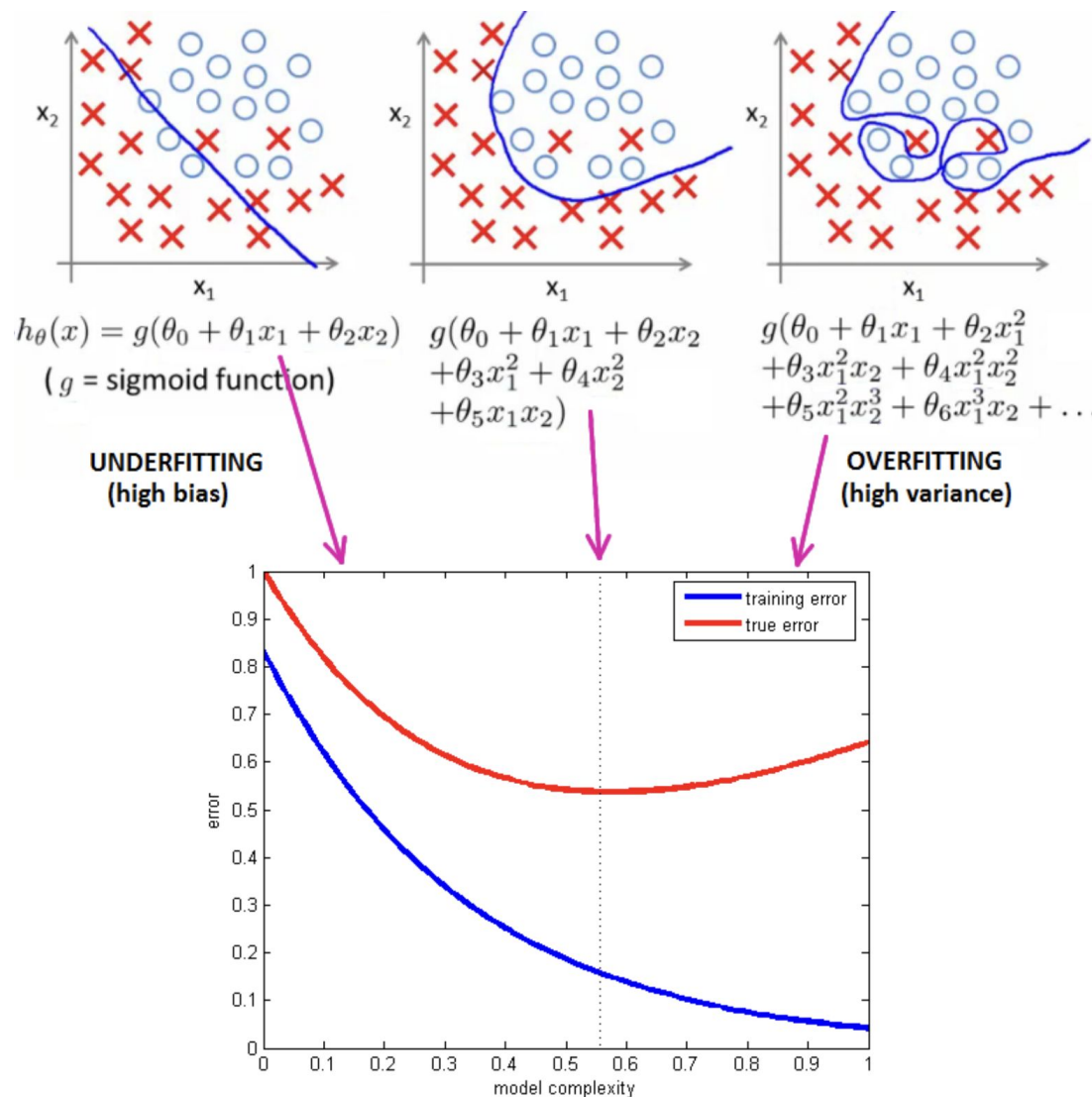




Overfitting and Regularization

Overfitting

- Overfitting occurs when the model learns the training data too well, resulting in poor performance on unseen data
- What indicates overfitting?
 - Performance on training set is much higher than on the test set
- How to prevent overfitting?
 - **Regularization**
 - **Cross-validation**
 - Increase training data
 - Early stopping
 - etc.



Methods to prevent overfitting by penalizing parameters.

- How it works

- Add a **penalty term** to the **loss function**

$$L = \sum_{i=1}^N \|y_i - f(\mathbf{x}_i)\|^2 + \lambda R(\mathbf{w})$$

Penalty term



- Encourage simpler models by penalizing large parameters

- Common methods

- **Ridge regression**
 - **Lasso regression**
 - ElasticNet

Ridge Regression

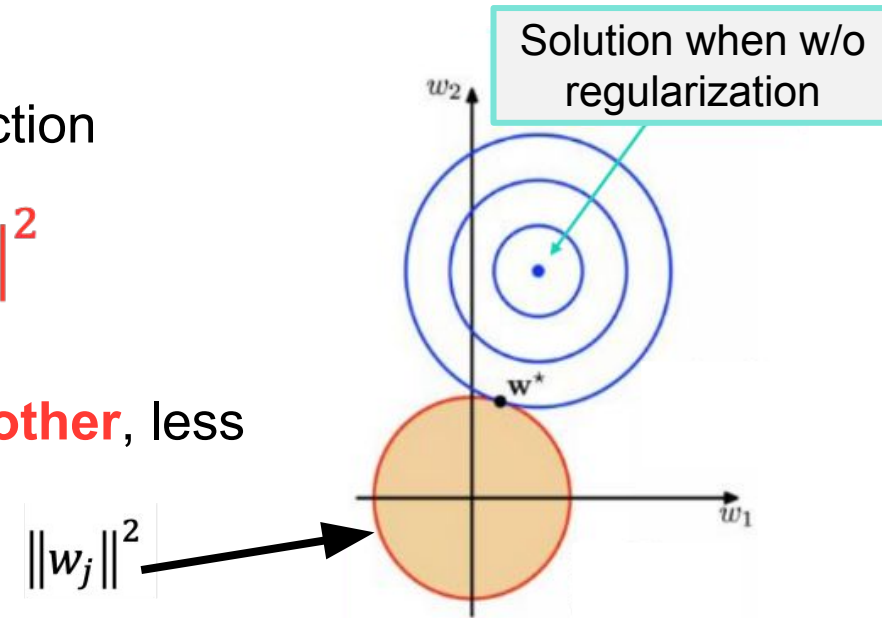
Penalizing weights of regression model using L2 norm.

- How it works

- Add a **L2 norm penalty term** to the loss function

$$L = \sum_{i=1}^N \|y_i - f(\mathbf{x}_i)\|^2 + \lambda \sum_{j=1}^M \|w_j\|^2$$

- Penalize larger weights and encourage **smoother**, less extreme model



Reference: C. Bishop "Pattern Recognition and Machine Learning"

Lasso Regression

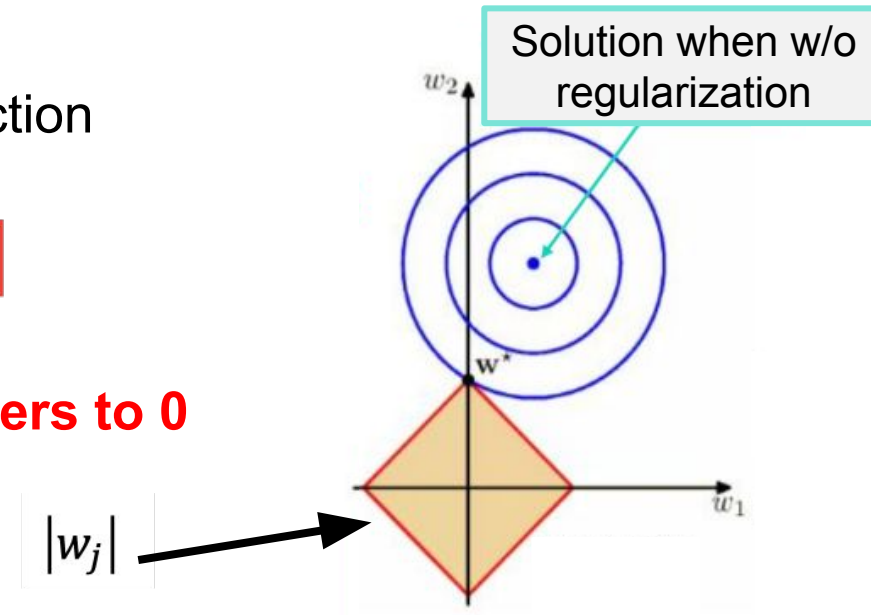
Penalizing weights of regression model using L2 norm.

- How it works

- Add a **L1 norm penalty term** to the loss function

$$L = \sum_{i=1}^N \|y_i - f(\mathbf{x}_i)\|^2 + \lambda \sum_{j=1}^M |w_j|$$

- Encourages the model to **set some parameters to 0**
 - Equivalent to removing some explanatory variables from the original data (= **shrinkage**)



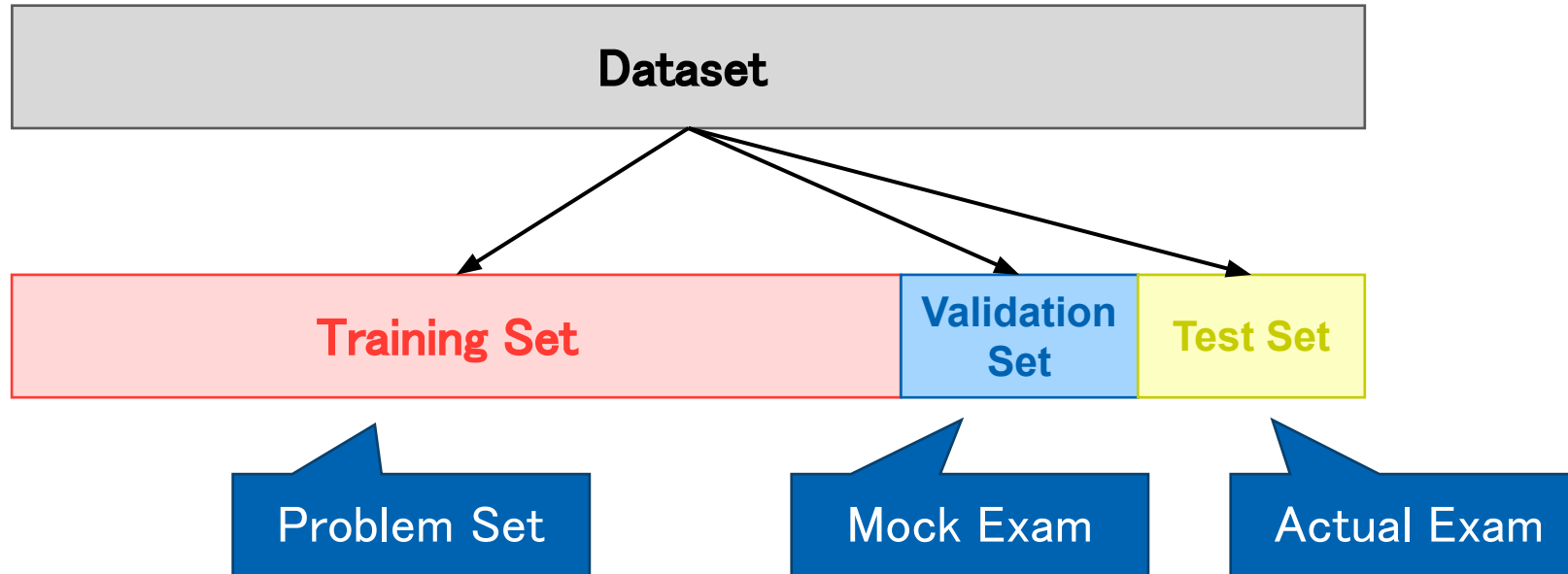
Reference: C. Bishop "Pattern Recognition and Machine Learning"

Evaluating Models

How to Evaluate the Model



- Split the data into three
 - **Training set**: Use to train the model
 - **Validation set**: Use to evaluate the model
 - **Test set**: Use to evaluate the best model



K-fold Cross-Validation

Create a validation set to evaluate the model's performance on unseen data, and then repeat the process.

Method

Split the data into k **splits**.
Use **k-1 splits as train set**, the **remaining one as validation set**.
Repeat k times.

- easy to implement
- reduces random biases

ex. 4-fold cross validation

#1	Val			
#2		Val		
#3			Val	
#4				Val

Use mean validation score

Stratified K-fold Cross-Validation

Perform cross-validation while preserving class proportions in each fold.

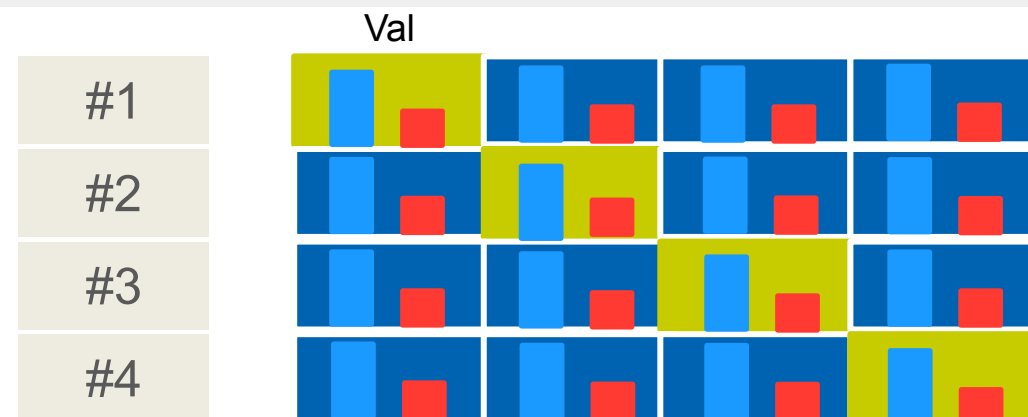
Method

Split the data into k splits, while **maintaining original class distribution**.

Use $k-1$ splits as train set, the remaining one as validation set.
Repeat k times.

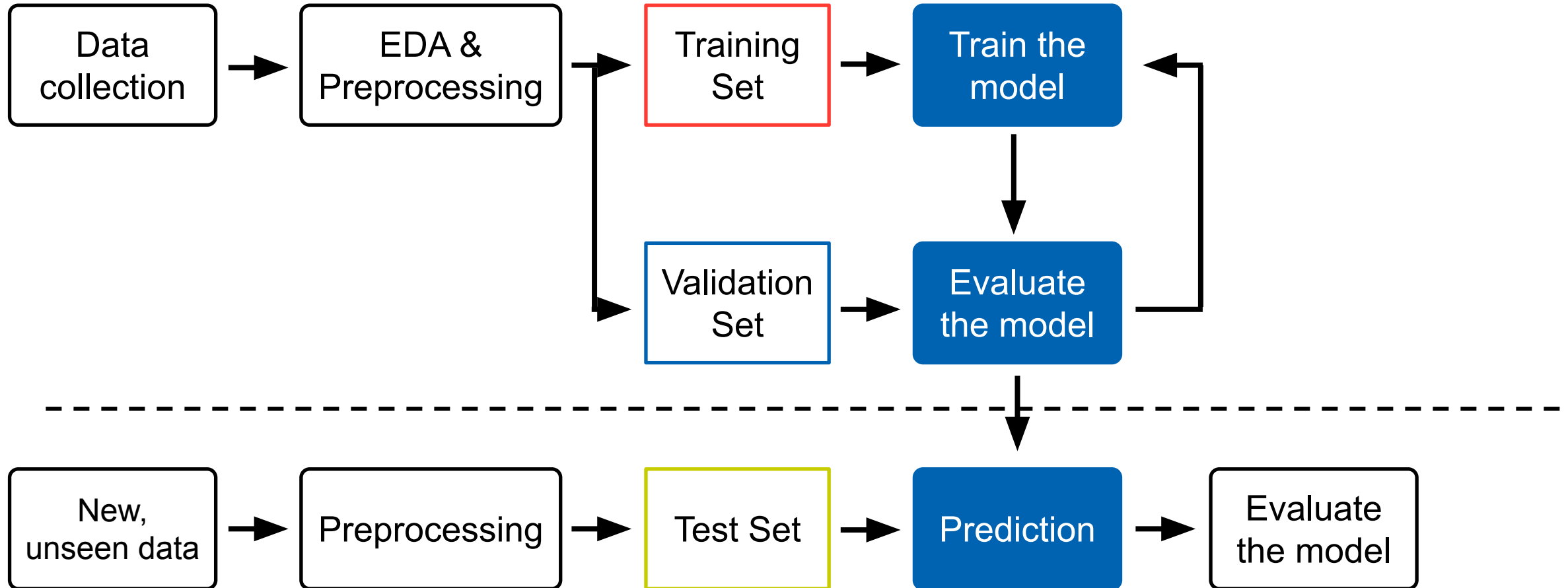
- Particularly effective for imbalanced datasets

ex. Stratified 4-fold cross validation



Use mean validation score

Supervised Learning Workflow (w/ cross-validation)



Hyperparameter Tuning

Hyperparameters are set of parameters of the model that are not learnable and needs to be set by humans beforehand.

Grid search evaluates every possible combination within the predefined range of the grid to search for the best hyperparameter for performance.

Pros

- Gives the best hyperparameter with in the given range of combination

Cons

- Computation takes time
 - Results in a increase of cost
- Can not explore a wide range of combination

Iteration	Hyperparam A	Hyperparam B	Hyperparam C
0	0	0	0
1	0	0	1
2	0	0	2
3	0	0	3
4	0	0	4
5	0	1	0
6	0	1	1
7	0	1	2
8	0	1	3
9	0	1	4

How to use GridSearchCV



```
from sklearn.model_selection import GridSearchCV
```

1. Import GridSearchCV from scikit-learn
(there are other types of searches as well)

```
# Prepare range of hyperparameters to search  
param_grid = {  
    "max_depth": [2, 3, 4, 5],  
    "criterion": ["entropy", "gini"]  
}
```

2. Prepare the hyperparameters to
search through as dictionary

```
# Create an instance for performing grid search  
gs = GridSearchCV(  
    estimator=DecisionTreeClassifier(), # Your choice of model  
    param_grid=param_grid,  
    cv=5 # Number of folds  
)
```

How to use GridSearchCV



```
from sklearn.model_selection import GridSearchCV

# Prepare range of hyperparameters to search
param_grid = {
    "max_depth": [2, 3, 4, 5],
    "criterion": ["entropy", "gini"]
}

# Create an instance for performing grid search
gs = GridSearchCV(
    estimator=DecisionTreeClassifier(), # Your choice of model
    param_grid=param_grid,
    cv=5 # Number of folds
)
```

=

```
cv = 5
for max_depth in [2, 3, 4, 5]:
    for criterion in ["entropy", "gini"]:
        model = DecisionTreeClassifier(
            max_depth=max_depth,
            criterion=criterion
        )

        # Do 5-fold cross validation
        ...
```

Supervised Learning Workflow (Summary)

